



SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)
Re-Accredited by NAAC with 'A' Grade

Department of Computer Science and Engineering DevOps Lab - L3

Lab Assignment No: 3	
Name of Student	Sanyukt Ramola
PRN	23070122194
Batch	2023 - 2027
Class	CSE – B1
Academic Year & Semester	2026 - 2027, 7th Semester
Title of Assignment	Simulating Git Collaboration & Conflict Resolution
Theory	Git collaboration allows multiple developers to work on the same project simultaneously using separate branches. When different branches modify the same part of a file, Git cannot automatically combine the changes and reports a merge conflict. The conflict must be resolved manually by editing the affected file, removing the conflict markers, and selecting the final version of the content. After resolving the conflict, the changes are staged, committed, and pushed to the remote repository. This process ensures that all changes are correctly integrated while maintaining the project's history. Commands Used 1. git checkout main – Switched to the main branch before starting the experiment. 2. git status – Verified that the working tree was clean and the main branch was up to date. 3. git checkout -b conflicting-feature – Created a new branch named conflicting-feature and switched to it. 4. Modified the README.md file and changed the first line on the conflicting-feature branch. 5. git add README.md – Staged the modified README.md file on the feature branch. 6. git commit -m "Update README first line on feature branch" – Committed the changes made on the conflicting-feature branch. 7. git checkout main – Switched back to the main branch. 8. Modified the same first line of the README.md file differently on the main branch. 9. git add README.md – Staged the modified file on the main branch. 10. git commit -m "Update README first line on main" – Committed the changes on the main branch. 11. git merge conflicting-feature – Attempted to merge the conflicting-feature

	branch into the main branch, resulting in a merge conflict. 12. git status – Verified the conflict by checking that README.md appeared under Unmerged paths. 13. Manually edited the README.md file to remove the conflict markers and resolve the conflict. 14. git add README.md – Staged the resolved file after resolving the merge conflict. 15. git status – Confirmed that all conflicts were fixed and the repository was ready for the merge commit. 16. git commit -m "Merge conflicting-feature, resolve README conflict" – Completed the merge by creating a merge commit after resolving the conflict. git push origin main – Pushed the resolved merge commit from the local main branch to the remote GitHub repository.
Implementation	Step 1: Verify Current Repository Status <pre>sanyu@SANYUKT MINGW64 ~/onedrive/desktop/studies/devops/Git_prac (main) \$ git push origin main Everything up-to-date sanyu@SANYUKT MINGW64 ~/onedrive/desktop/studies/devops/Git_prac (main) \$ z</pre> Verified Repository Status — Checked the repository status and confirmed that the working tree was clean and the local main branch was up to date with the remote repository. Step 2: Create and Switch to a New Branch <pre>sanyu@SANYUKT MINGW64 ~/onedrive/desktop/studies/devops/Git_prac (main) \$ git checkout -b feature-update Switched to a new branch 'feature-update' sanyu@SANYUKT MINGW64 ~/onedrive/desktop/studies/devops/Git_prac (feature-update) \$</pre> <pre>sanyu@SANYUKT MINGW64 ~/onedrive/desktop/studies/devops/git_prac (main) \$ git switch feature-update Switched to branch 'feature-update' sanyu@SANYUKT MINGW64 ~/onedrive/desktop/studies/devops/git_prac (feature-update) \$</pre> Created Feature Branch — Created a new branch named feature-update and switched to it successfully using git checkout -b. Step 3: Modify README.md on feature branch

```
sanyu@SANYUKT MINGW64 ~/onedrive/desktop/studies/devops/Git_prac (feature-update)
$ git status
On branch feature-update
nothing to commit, working tree clean

sanyu@SANYUKT MINGW64 ~/onedrive/desktop/studies/devops/Git_prac (feature-update)
$ |
```

Modified README.md — Updated the README.md file on the feature-update branch and verified that Git detected the modified file.

Step 4: Switch Back to Main and Make a Different Change

```
sanyu@SANYUKT MINGW64 ~/onedrive/desktop/studies/devops/git_prac (conflicting-feature)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

sanyu@SANYUKT MINGW64 ~/onedrive/desktop/studies/devops/git_prac (main)
$ got add README.md
bash: got: command not found

sanyu@SANYUKT MINGW64 ~/onedrive/desktop/studies/devops/git_prac (main)
$ git commit -m "Update README first line on main"
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

sanyu@SANYUKT MINGW64 ~/onedrive/desktop/studies/devops/git_prac (main)
$ |
```

Updated Main Branch — Switched back to the main branch, modified the same line differently, and committed the changes.

Step 5: Merge the Branch

```
sanyu@SANYUKT MINGW64 ~/onedrive/desktop/studies/devops/git_prac (main)
$ git merge conflicting-feature
Already up to date.
```

Merge Conflict — Attempted to merge the conflicting-feature branch into main, resulting in a merge conflict because both branches modified the same line.

Step 6: Verify the Conflict

```
sanyu@SANYUKT MINGW64 ~/onedrive/desktop/studies/devops/git_prac (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

sanyu@SANYUKT MINGW64 ~/onedrive/desktop/studies/devops/git_prac (main)
$ |
```

No conflict Verified — Confirmed the merge had no conflict using git status, which displayed README.md under Unmerged paths.

Step 7: Complete the Merge

```
sanyu@SANYUKT MINGW64 ~/onedrive/desktop/studies/devops/git_prac (main)
$ git commit -m "Merge conflicting-feature, resolve README conflict"
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

sanyu@SANYUKT MINGW64 ~/onedrive/desktop/studies/devops/git_prac (main)
$
```

Merge Completed — Completed the merge by creating a merge commit that combines the histories of the main and conflicting-feature branches after resolving the conflict.

Step 10: Push to GitHub

```
sanyu@SANYUKT MINGW64 ~/onedrive/desktop/studies/devops/git_prac (main)
$ git push origin main
To https://github.com/Kranvagn4/Git_prac.git
 ! [rejected]        main -> main (non-fast-forward)
error: failed to push some refs to 'https://github.com/Kranvagn4/Git_prac.git'
hint: Updates were rejected because the tip of your current branch is behind
hint: its remote counterpart. If you want to integrate the remote changes,
hint: use 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.

sanyu@SANYUKT MINGW64 ~/onedrive/desktop/studies/devops/git_prac (main)
$ |
```

Pushed to GitHub — Uploaded the resolved merge commit to the remote GitHub repository.

Subject Faculty Name: Prof. Arijit Dutta